Detailed Methodology for Condition-Aware Latent State-of-Health Estimation, Forecasting, and Operational Translation in Electric Aircraft Batteries

Ben Fogerty Chintan Desai Shailleze Mahenrenathan
Sher Verma

Sneh Shah
MSE 402 Capstone, Group 18

June 4, 2026

Abstract

This document is the methods companion to the Group 18 capstone Final Design Review. Its purpose is to explain, with equations and reasoning, how the project converts noisy battery-management-system (BMS) telemetry into a usable battery health signal, how that signal is forecast over short and long horizons, and how it is translated into operational quantities for an advisory planner. Each section first explains *why* the design choice was made, then states the equations that implement it. Reported numerical results match the artifacts written by the committed pipeline.

Contents

1 Problem Framing **1**

2 From Telemetry to Events **2**
2.1 Event Type Comes From SOC Direction, Not Labels 2
2.2 Per-Event Summary Features 2

3 A Capacity-Based Sanity Check on SOH **2**

4 The Latent SOH Model **3**
4.1 Why a Kalman Filter 3
4.2 The State-Space Model 3
4.3 How Much Drift Is Allowed Between Events: Q_k 4
4.4 How Much to Trust Each Measurement: R_k 4
4.5 The Kalman Filter Equations 5
4.6 The Smoother (for Reports, Not for Training) 5
4.7 Result 6

5 Forecasting Latent SOH **6**
5.1 Why a Sequence Model 6
5.2 Target and Inputs 6
5.3 Predicting a Change, Not a Level 7

5.4 The Recurrent Cells . . . 7
5.5 Loss and Optimization . . . 8
5.6 Validation: Why Walk-Forward . . . 8
5.7 Result . . . 8

6 Physics-Informed Forecasters **8**
6.1 Why Add Physics . . . 8
6.2 Aging Channels . . . 9
6.3 The Hybrid Model . . . 9
6.4 The Standard PINN . . . 10

7 Long-Horizon Outlook via an External Backbone **10**
7.1 Why We Cannot Forecast EOL From Our Data Alone . . . 10
7.2 The Backbone Shape . . . 11
7.3 Placing an Aircraft Battery on the Backbone . . . 11

8 From SOH to Operational Quantities **11**
8.1 Circuits per Flight . . . 12
8.2 SOC Burn Rate . . . 12

9 Putting It All Together **13**

10 Limitations **13**

11 Why Each Choice Was Made **13**

1 Problem Framing

The Pipistrel VELIS Electro reports a battery state of health (SOH) on every flight and charge event. In principle, this value should gradually decline as the battery ages. In practice, the reported value jumps around: it can increase by tens of percentage points between back-to-back events, behave very differently on charging events than on flights, and shift whenever the BMS recalibrates its internal capacity estimate. Treating that raw number as ground truth would mean training a model to predict noise.

The project therefore separates three quantities:

- z_k: the raw BMS-reported SOH at event k. Useful, but noisy and context-dependent.
- x_k: a latent SOH state, recovered by a Kalman filter that treats z_k as a noisy observation of x_k. This is the signal we trust.
- $\widehat{y}_{k,h}$: a forecast of latent SOH h flights ahead of event k, produced by a learned model trained on the latent label.

Throughout the report, an event is one operational segment for one battery on one aircraft (a flight, a charge, or an idle gap). Events are ordered chronologically per battery stream. The committed dataset contains 1,204 events: 1,106 from Plane 166 and 98 from Plane 192, used as a small out-of-distribution holdout.

2 From Telemetry to Events

The pipeline starts by reducing each event's raw sample-by-sample telemetry to a single row of summary features. Two design decisions matter for everything that follows.

2.1 Event Type Comes From SOC Direction, Not Labels

The raw event labels in the data are not always correct. Short ground operations are sometimes tagged as flights, and a few charge sessions are tagged ambiguously. Because the downstream noise model treats charge events differently from flight events, we rebuild the event type from the data itself.

For each event, we look at how the battery's SOC changed from start to end. Let ΔSOC be the median SOC over the last samples of the event minus the median SOC over the first samples. With a small threshold τ to ignore noise,

$$\text{event type} = \begin{cases} \texttt{charge}, & \Delta\text{SOC} \geq \tau, \\ \texttt{flight}, & \Delta\text{SOC} \leq -\tau, \\ \texttt{idle}, & |\Delta\text{SOC}| < \tau. \end{cases} \tag{1}$$

A charge event has SOC going up; a flight (or ground discharge) has SOC going down; an idle gap has SOC essentially flat. This is the only classification the rest of the pipeline depends on.

2.2 Per-Event Summary Features

Within each event, every sample is checked against physical limits (a plausible current magnitude, voltage range, and temperature range for that event type). Samples that violate those limits are dropped, then the event is summarized by a fixed set of statistics:

- the median observed SOH z_k,
- the SOH inter-quartile range $\text{IQR}_{\text{SOH},k}$, a measure of within-event SOH instability,
- mean and 95^{th}-percentile current, temperature, and voltage,
- SOC minimum, maximum, mean, and span,
- the rate-of-change of current and temperature ($|dI/dt|$, $|dT/dt|$), summarized as their 95th percentiles,
- the gap between the BMS's Kalman SOC estimate and a coulomb-counting SOC estimate (this gap grows when the BMS is losing track of true SOC).

These features are the inputs to both the noise model and the forecasting models. The reason they matter is that they tell us how much we should trust the SOH reading at this event.

3 A Capacity-Based Sanity Check on SOH

Before introducing the latent model, it is worth showing why we cannot just compute SOH from physics directly.

On a clean charging event, you can integrate the charging current to get the total charge delivered, then divide by the SOC range covered to estimate the battery's full capacity:

$$Q_{\text{delivered}} = \int |I(t)|\, dt/3600 \quad \text{(in Ah)}, \qquad \widehat{C}_{\text{event}} = \frac{Q_{\text{delivered}}}{(\text{SOC}_{\text{hi}} - \text{SOC}_{\text{lo}})/100}. \tag{2}$$

A physics-based SOH is then $100\,\widehat{C}/C_{\text{rated}}$. The partial-charging variant of this idea is well-developed in the literature: Jenu et al. [9] demonstrate that integrated-voltage and incremental-capacity methods can give SOH error below 2% RMSE on cycled lithium-ion cells under controlled C-rates, and Saini and Renganathan [10] extend incremental capacity analysis to dynamic-load electric-vehicle data.

This is a clean idea, but in practice the estimate is noisy on the aircraft telemetry. On Plane 166, comparing this charge-physics SOH against the BMS SOH gives MAE ≈ 7.9 SOH points and $R^2 \approx 0.11$: the two agree in direction but not closely. We keep this estimator as a cross-check, but not as the supervised target. A more robust approach is to treat the BMS reading as a noisy measurement of an underlying true SOH, and infer the true SOH from many measurements over time.

4 The Latent SOH Model

4.1 Why a Kalman Filter

Battery aging is gradual: real SOH drifts down slowly over months. The problem is not that we lack measurements; the problem is that any single measurement may be wrong by several SOH points because of the operating context (charging, high current, edge-of-SOC, BMS recalibration). What we want is a way to combine many noisy measurements over time into a stable estimate.

This is exactly what a Kalman filter does. It maintains a running estimate of a hidden quantity, updates that estimate as new measurements arrive, and weights each measurement by how much we trust it. If a measurement looks reliable, the estimate moves toward it. If a measurement looks unreliable, the estimate stays close to its prior.

Treating battery state as a hidden quantity recovered through Kalman filtering is well established in the battery-management literature. The foundational treatment by Plett [1, 2, 3] introduced extended Kalman filtering for joint state-of-charge and state-of-health estimation in lithium-ion battery packs, and subsequent work has consistently treated SOH as a slowly varying hidden state inferred from noisy terminal measurements [7]. The mathematical foundations of optimal filtering and smoothing used in this report follow the standard treatment in Särkkä and Svensson [6].

4.2 The State-Space Model

We model latent SOH as a scalar quantity that drifts slowly over time:

$$x_k = x_{k-1} + w_k, \qquad w_k \sim \mathcal{N}(0, Q_k) \quad \text{(process)} \tag{3}$$

$$z_k = x_k + v_k, \qquad v_k \sim \mathcal{N}(0, R_k) \quad \text{(observation)} \tag{4}$$

In words: between events, the latent SOH changes by a small random amount (Gaussian, mean zero, variance Q_k). At each event, the BMS reports a measurement z_k that equals the true latent SOH plus a Gaussian observation error with variance R_k.

This is the simplest possible state-space model. We deliberately do not model degradation rate as a second state, because we do not have enough events per battery to estimate a rate cleanly. Instead, the rate emerges from how the latent state drifts over time. The random-walk treatment of SOH as a slowly drifting hidden parameter is the standard battery-EKF formulation in Plett [3] and is reflected in the recent review by Yang et al. [7].

4.3 How Much Drift Is Allowed Between Events: Q_k

Real degradation happens on a calendar time scale, not a per-event time scale. A long idle gap between flights gives the battery more time to degrade than two back-to-back events. We therefore tie the process variance to the elapsed time:

$$Q_k \;=\; \sigma_q^2 \, \Delta t_k, \tag{5}$$

where Δt_k is the time between events $k-1$ and k in days and σ_q is the allowed drift per square-root-day. We use $\sigma_q = 0.1$ SOH points per $\sqrt{\text{day}}$. The choice is deliberately tight: we want the latent state to change slowly, so that real degradation can drive it down over weeks but a single noisy measurement cannot push it around.

4.4 How Much to Trust Each Measurement: R_k

This is the most important design decision in the project. A fixed measurement variance would tell the filter to trust every BMS reading equally. But we know from the data that some readings are far more reliable than others. A reading taken mid-flight at moderate current and stable temperature is trustworthy. A reading taken at the top of charge, right after a BMS recalibration, is not.

The condition-aware approach is to inflate R_k on events that look operationally extreme. The filter will then give those events less influence on the latent state. This idea — a Kalman filter with time-varying or adaptive measurement-noise covariance — has a long history in the estimation literature, beginning with Mehra's classification of adaptive filtering approaches [5], and is the basis for modern adaptive Kalman variants used in lithium-ion battery state estimation under variable operating conditions [7]. The distinction in our approach is that, rather than adapting R_k from innovation residuals after the fact, we set it *ahead of time* from interpretable physical condition scores, which makes the noise model transparent and auditable.

We compute several bounded "stress" scores for each event, each between roughly 0 (calm) and 2 (extreme):

Score	What it measures
s_k^{current}	How close 95th-percentile current is to the rated maximum
$s_k^{dI/dt}$	How quickly current was changing during the event
$s_k^{dT/dt}$	How quickly temperature was changing during the event
s_k^{SOCedge}	How close the event ran to SOC = 0% or 100%
s_k^{instab}	How variable the SOH reading itself was during the event
s_k^{gap}	Disagreement between the BMS's Kalman SOC and coulomb-counted SOC
s_k^{switch}	Whether a BMS recalibration flag fired during the event
s_k^{evtype}	Larger on charge events than on flights (charges are noisier)

The general form of each score is a clipped ratio, for example

$$s_k^{\text{current}} \;=\; \text{clip}\!\left(\frac{\text{p95}|I|_k}{I_{\max}},\, 0,\, 2\right), \qquad s_k^{dI/dt} \;=\; \text{clip}\!\left(\frac{\log\!\big(1+\text{p95}|dI/dt|_k\big)}{\log(1+20)},\, 0,\, 2\right). \tag{6}$$

The exact upper limits and the use of $\log(1+\cdot)$ for the rate scores are just to keep all scores on a comparable scale; another implementer could pick different normalizations.

A weighted sum of these scores defines a condition multiplier:

$$m_k \;=\; 1 + \sum_j w_j \, s_k^{(j)}. \tag{7}$$

A baseline "calm event" has all scores at zero and $m_k = 1$. An extreme event has m_k several units larger. The weights w_j are chosen so that signals we believe more strongly (e.g. within-event SOH variability, BMS recalibration flags) inflate the multiplier more.

Finally, we anchor the multiplier to a per-stream baseline standard deviation σ_{base}, estimated robustly from event-to-event SOH changes on the calmest events using the median absolute deviation. The measurement variance is

$$R_k \;=\; \big(\sigma_{\text{base}}\, m_k\big)^2, \qquad \sigma_{\text{base}} \;\approx\; \frac{1.4826\,\text{MAD}(\Delta z)}{\sqrt{2}}. \tag{8}$$

The factor 1.4826 converts MAD to an equivalent Gaussian standard deviation; the $\sqrt{2}$ accounts for the fact that Δz is a difference of two noisy observations. The result is that R_k is small on calm events (the filter trusts them) and large on extreme events (the filter discounts them).

4.5 The Kalman Filter Equations

With latent SOH as a scalar, the filter is especially simple. For each event $k = 1, 2, \ldots$:

$$\widehat{x}_{k|k-1} = \widehat{x}_{k-1|k-1} \qquad \text{(predict the state)} \tag{9}$$

$$P_{k|k-1} = P_{k-1|k-1} + Q_k \qquad \text{(predict its variance)} \tag{10}$$

$$K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R_k} \qquad \text{(Kalman gain)} \tag{11}$$

$$\widehat{x}_{k|k} = \widehat{x}_{k|k-1} + K_k\,(z_k - \widehat{x}_{k|k-1}) \qquad \text{(update with measurement)} \tag{12}$$

$$P_{k|k} = (1 - K_k)\,P_{k|k-1} \qquad \text{(update its variance)} \tag{13}$$

The Kalman gain K_k is the key quantity. It is a number between 0 and 1. When $R_k \gg P_{k|k-1}$ the gain is near 0 and the filter essentially ignores the measurement. When $R_k \ll P_{k|k-1}$ the gain is near 1 and the filter moves the estimate toward the measurement. The condition-aware R_k is what controls this balance event by event.

The output $\widehat{x}_{k|k}$ is the *causal* latent SOH at event k, based only on past and current measurements. This is what we use as the supervised target for forecasting models, because future information cannot leak into it.

4.6 The Smoother (for Reports, Not for Training)

After the forward filter has passed through the entire history, we can also run a backward pass that revisits each event with the benefit of future evidence. The standard Rauch–Tung–Striebel (RTS) recursion, originally derived for maximum-likelihood estimation in linear dynamic systems [4], is

$$G_k = \frac{P_{k|k}}{P_{k+1|k}}, \tag{14}$$

$$\widehat{x}_{k|N} = \widehat{x}_{k|k} + G_k\,\big(\widehat{x}_{k+1|N} - \widehat{x}_{k+1|k}\big), \tag{15}$$

$$P_{k|N} = P_{k|k} + G_k^2\big(P_{k+1|N} - P_{k+1|k}\big). \tag{16}$$

The smoothed estimate $\widehat{x}_{k|N}$ is the best retrospective look at the latent trajectory: it uses every event, not just events up to k. It is what we plot when we want to visualize how SOH actually evolved. We never use it as a training target, because it knows about the future and would leak that information into the forecaster.

4.7 Result

The model achieves the central goal: it removes the unphysical jumps in the raw BMS signal while preserving the long-run trend. On Plane 166:

Metric	Raw SOH	Latent SOH
Total event-to-event variation (Battery 1)	484	47
Total event-to-event variation (Battery 2)	469	45
Largest single upward jump (Battery 1)	29	0.7
Largest single upward jump (Battery 2)	25	0.7

The raw signal can apparently "recover" twenty-nine SOH points in a single event. The latent signal cannot, by construction, because the condition-aware filter recognized those jumps as coming from unreliable measurements.

5 Forecasting Latent SOH

Once we have a stable latent SOH, the next question is how it will evolve. We frame this as: given the latent SOH at event k and the event history leading up to it, predict the latent SOH h flights into the future.

5.1 Why a Sequence Model

Battery degradation depends on history, not just on the current state. Two batteries with the same SOH today can degrade at very different rates depending on how they have been used — frequent fast charges, high temperatures, deep discharges, idle storage at high SOC, and so on. A model that only sees today's state cannot tell those two batteries apart.

Recurrent neural networks (LSTM and GRU) are designed exactly for this: they consume a sequence of past events and produce a single prediction that depends on the whole sequence. LSTM-based SOH estimators have been demonstrated in the battery-health literature with mean-absolute percentage errors below 2.22% on capacity-test data [8], which motivated our choice of recurrent models as the primary forecasting family. We use them here to predict SOH h flights ahead.

5.2 Target and Inputs

For each event k, the target is the latent SOH at the future event that lands h flights later:

$$y_{k,h} \;=\; x^{\text{filter}}_{j(k,h)}, \qquad j(k,h) \;=\; \min\{j > k \,:\, f_j \geq f_k + h\}, \tag{17}$$

where f_k is the cumulative flight count for that battery up to event k. Using cumulative flight count rather than calendar time ensures that charge events and idle gaps do not distort the horizon.

The model input at event k is a window of the last L events, each represented by a feature vector $\mathbf{u}_t$ containing the summary statistics from Section 2.2 plus engineered features (cumulative

throughput, rolling averages, lagged SOH, etc.). Concretely,

$$\mathbf{X}_k \;=\; \begin{bmatrix} \mathbf{u}_{k-L+1} \\ \mathbf{u}_{k-L+2} \\ \vdots \\ \mathbf{u}_k \end{bmatrix} \in \mathbb{R}^{L\times p}. \tag{18}$$

Features are standardized using training-split statistics so that the network sees comparable scales across columns.

5.3 Predicting a Change, Not a Level

A useful trick is to ask the network to predict the *change* in SOH rather than the next SOH directly:

$$\widehat{y}_{k,h} \;=\; x_k^{\text{filter}} + g_\theta(\mathbf{X}_k). \tag{19}$$

The network output $g_\theta(\mathbf{X}_k)$ is a small delta, and adding it back to the current SOH gives the level. This focuses the network's capacity on the part that actually requires learning — incremental degradation — rather than on relearning the level of SOH on every event.

5.4 The Recurrent Cells

We use LSTM and GRU as the recurrent core. Both consume the lookback window step by step and produce a final hidden state that summarizes the sequence. For one step t, the LSTM cell is

$$\mathbf{i}_t = \sigma(\mathbf{W}_i\mathbf{u}_t + \mathbf{U}_i\mathbf{h}_{t-1} + \mathbf{b}_i), \qquad \text{input gate} \tag{20}$$
$$\mathbf{f}_t = \sigma(\mathbf{W}_f\mathbf{u}_t + \mathbf{U}_f\mathbf{h}_{t-1} + \mathbf{b}_f), \qquad \text{forget gate} \tag{21}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{u}_t + \mathbf{U}_o\mathbf{h}_{t-1} + \mathbf{b}_o), \qquad \text{output gate} \tag{22}$$
$$\widetilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c\mathbf{u}_t + \mathbf{U}_c\mathbf{h}_{t-1} + \mathbf{b}_c), \qquad \text{candidate cell} \tag{23}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \widetilde{\mathbf{c}}_t, \qquad \text{cell state} \tag{24}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \qquad \text{hidden state} \tag{25}$$

The three sigmoidal gates control how much past information is retained, how much new information is admitted, and how much of the internal state is exposed. The final hidden state $\mathbf{h}_L$ is passed through a small multilayer perceptron head to produce the scalar delta prediction.

The GRU is a simpler variant of the same idea, using two gates instead of three:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z\mathbf{u}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z), \qquad \text{update gate} \tag{26}$$
$$\mathbf{r}_t = \sigma(\mathbf{W}_r\mathbf{u}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r), \qquad \text{reset gate} \tag{27}$$
$$\widetilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h\mathbf{u}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h), \tag{28}$$
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \widetilde{\mathbf{h}}_t. \tag{29}$$

We try both because GRUs are sometimes easier to train on small datasets while LSTMs sometimes hold longer dependencies more stably. The winning model is chosen by validation MAE at each horizon.

5.5 Loss and Optimization

Training minimizes the Smooth L1 (Huber) loss on the delta target:

$$\mathcal{L}(\theta) \;=\; \frac{1}{N}\sum_{k=1}^{N} \mathrm{SmoothL1}\big(g_\theta(\mathbf{X}_k),\; y_{k,h} - x_k^{\mathrm{filter}}\big), \tag{30}$$

where

$$\mathrm{SmoothL1}(\widehat{u}, u) = \begin{cases} \tfrac{1}{2}(\widehat{u}-u)^2, & |\widehat{u}-u| < 1, \\ |\widehat{u}-u| - \tfrac{1}{2}, & \text{otherwise.} \end{cases} \tag{31}$$

Huber loss is used in preference to plain MSE because it is less sensitive to a small number of large residuals, which is helpful given the noisiness of the underlying problem. We optimize with Adam, gradient clipping, and early stopping on validation MAE.

5.6 Validation: Why Walk-Forward

A standard random train/test split would let the model peek at events from the future of its training history, which is exactly the kind of leakage we are trying to avoid. We use a chronological walk-forward scheme instead: train on the earliest fraction of events, validate on the next contiguous chunk, then advance. Plane 192 is kept entirely out of training and used only to check that the model has not overfitted to Plane 166.

5.7 Result

Forecast quality depends strongly on horizon. The committed best models and their backtest mean absolute errors are:

Horizon (flights ahead)	Best model	Backtest MAE (SOH points)
1	LSTM	0.19
5	LSTM	0.66
10	GRU	1.14
15	GRU	1.17
20	GRU	1.19

Errors at one and five flights are well within useful precision for operational planning. Errors at fifteen and twenty flights are about six times larger than at one flight. This is a meaningful and honest result: the data supports confident short-horizon prediction, but not confident long-horizon prediction. The long-horizon problem requires a different approach.

6 Physics-Informed Forecasters

Two of the trained models add physical structure to the recurrent forecaster. They are useful both for performance and for interpretability.

6.1 Why Add Physics

A pure recurrent network is a flexible function approximator. With limited data, that flexibility can lead it to fit patterns that are not physically meaningful: SOH might be predicted to recover, or

to drop faster on a cool flight than on a hot one. Physics-informed structure constrains the model to behave like a battery: degradation only goes one way, certain stresses (heat, throughput, time) increase degradation, and the degradation rate should change smoothly from one similar event to the next.

We use two such models: a *hybrid* model that hard-codes battery aging intuition into its architecture, and a *standard PINN* that enforces a differential equation through automatic differentiation. The PINN formulation is inspired by Gu et al. [11], who parameterize a single-particle electrochemical model with a physics-informed neural network and identify internal aging parameters from partial charging profiles. Our PINN does not solve the full electrochemical PDE — our event-level data does not support that — but it borrows the idea of embedding a physical degradation law into the training loss.

6.2 Aging Channels

Both physics-informed models decompose degradation into five physically named drives, each a nonnegative scalar built from the event summary features. The intuition behind each:

- D_k^{cal} (calendar): captures aging that happens simply because time passed. Larger on long idle gaps, especially at high storage SOC.
- D_k^{cyc} (cycling): captures aging from throughput. Larger on long, high-current discharges with wide SOC swings.
- D_k^{th} (thermal): captures aging from heat. Built from an Arrhenius-style temperature factor and time spent above warm temperatures.
- D_k^{res} (resistance/health-fault): captures aging signals from voltage sag and from inconsistency between the BMS's own SOC estimators.
- D_k^{hist} (history): captures accumulated cumulative stress, including total equivalent full cycles and rolling stress.

The actual definitions are smooth nonnegative combinations of upstream features wrapped in $\log(1+\cdot)$ to keep them on a comparable scale. A different team could choose somewhat different formulas; the important property is that all five drives are nonnegative and that they correspond to distinct aging mechanisms.

6.3 The Hybrid Model

The hybrid model uses the aging channels directly in its prediction rule. A small neural network looks at the event context and produces five nonnegative sensitivities $\alpha_{k,1}, \ldots, \alpha_{k,5}$, one per channel. The base degradation at event k is then the dot product

$$\mathcal{D}_k^{\text{base}} \;=\; \sum_{j=1}^{5} \alpha_{k,j}\, D_k^{(j)}. \tag{32}$$

Two additional terms refine this: a small interaction term that lets channels reinforce one another (high temperature *and* high throughput is worse than either alone), and a learned residual term that captures whatever the explicit channels miss. The total degradation is subtracted from the current SOH to give the prediction:

$$\widehat{x}_{k+1} \;=\; \text{clip}\big(x_k^{\text{filter}} - \mathcal{D}_k,\; 0,\; 100\big). \tag{33}$$

Two structural properties matter:

- The sensitivities $\alpha_{k,j}$ are forced to be nonnegative (via a softplus), so degradation cannot become negative and SOH cannot "recover" through the explicit channels.
- The sensitivities are encouraged to vary *smoothly* between nearby events on the same battery. The training loss includes a penalty on $\|\alpha_k - \alpha_{k-1}\|^2$, weighted by an exponential decay over the inter-event gap. This means the same battery cannot suddenly change its aging law from one event to the next.

6.4 The Standard PINN

The PINN takes a more classical approach. It treats SOH as a continuous function of battery age t, $u_\theta(t)$, and asks the network to satisfy a governing differential equation:

$$\frac{du_\theta}{dt} \;+\; g(\mathbf{D}_k,\, u_\theta) \;=\; 0, \tag{34}$$

where $g(\cdot)$ is a nonnegative degradation rate built from the same aging channels and a small set of trainable weights:

$$g(\mathbf{D}, u) \;=\; \left(b + \mathbf{w}^\top \mathbf{D}\right)\left(1 + \gamma\, \tfrac{100-u}{100}\right), \qquad b, \mathbf{w}, \gamma \geq 0. \tag{35}$$

Reading the equation: the rate of SOH loss is positive, increases with the aging drives, and increases as the battery moves further from fresh.

The clever part is how the PINN is trained. We do not need to discretize the ODE; PyTorch's automatic differentiation gives us du_θ/dt directly. We then add a residual loss that penalizes the network whenever $du_\theta/dt + g(\mathbf{D}, u_\theta)$ departs from zero. The full training objective combines:

- a data loss matching $u_\theta(t_k)$ to observed SOH,
- a step loss matching the Euler-stepped forecast to observed next SOH,
- the ODE-residual loss above,
- a monotonic prior penalizing any positive du_θ/dt (SOH should not go up over time),
- an initial-condition term anchoring the first event.

The PINN is more conceptually faithful to the underlying physics than the hybrid model, but it is harder to train on the available dataset because the residual loss must be balanced against the other terms.

7 Long-Horizon Outlook via an External Backbone

7.1 Why We Cannot Forecast EOL From Our Data Alone

The aircraft batteries in the dataset have not yet aged to the end of their life. We have observed perhaps thirty percent of a typical battery's lifetime; we have never seen one reach the replacement threshold. Any model that tries to extrapolate to end-of-life from our data alone is extrapolating beyond its training range.

The fix is to borrow the *shape* of degradation from an external dataset that does cover full battery lifetimes, then place our batteries on that shape.

7.2 The Backbone Shape

We use the public eVTOL battery test dataset from Bills et al. [12] (Carnegie Mellon University), which contains detailed cycle-life experiments on lithium-ion cells operated under representative eVTOL mission profiles. The dataset spans many cells cycled to end of life under varying cruise lengths, charge currents, voltage limits, and temperatures, making it well suited for learning a generic eVTOL degradation shape. For each test battery, we map calendar progress to normalized progress $p \in [0, 1]$, with $p = 0$ at the start of life and $p = 1$ at end of life. We also map measured capacity to an adjusted health scale where 100 corresponds to fresh and 0 corresponds to the end-of-life capacity threshold.

Pooling all external test points gives a cloud of $(p_a, \text{Health}_a^{\text{adj}})$ pairs. We fit a monotone-decreasing function $B(p)$ through this cloud using isotonic regression:

$$B(p) \;=\; \arg \min_{\substack{f:[0,1]\to[0,100] \\ f \text{ non-increasing}}} \sum_a \big(\text{Health}_a^{\text{adj}} - f(p_a)\big)^2. \tag{36}$$

The result is a smooth, decreasing curve that captures the typical degradation trajectory of an eVTOL battery from new to retired.

7.3 Placing an Aircraft Battery on the Backbone

For each of our aircraft batteries, we estimate where on the backbone the battery currently sits. We grid-search two quantities:

- p_0: the normalized progress at which the battery's observable life starts (because we do not have data from new).
- L: the total life of the battery in flights.

For each candidate pair, we predict latent SOH at every observed flight using the backbone, then minimize the mean absolute error:

$$(p_0^\star, L^\star) \;=\; \arg\min_{p_0, L} \frac{1}{n} \sum_{k=1}^{n} \left| x_k^{\text{filter}} - B\Big(p_0 + \tfrac{f_k - f_1}{L}\Big) \right|. \tag{37}$$

Once the best $(p_0^\star, L^\star)$ is known, the remaining flights to end of life are simply $(1 - p_{\text{end}}^\star)\, L^\star$, where $p_{\text{end}}^\star$ is the progress at the most recent event.

For Plane 166, the fitted MAE between the calibrated backbone and the latent SOH trajectory is approximately 1.7 SOH points across both batteries, which means the external shape genuinely captures the long-run behavior of our aircraft batteries even though we have not observed them anywhere near end of life.

8 From SOH to Operational Quantities

The final piece of the pipeline turns SOH into things an operator actually plans around: how many circuits the aircraft can fly today, how fast the battery will drain on a given flight, and when the battery will need to be replaced.

8.1 Circuits per Flight

The Pilot Operating Handbook gives a baseline relationship between SOH and SOC consumed per traffic circuit, in the form of a small lookup table. We turn this into a continuous function by linear interpolation:

$$s_{\mathrm{POH}}(\mathrm{SOH}) \;=\; \mathrm{interp}\big(\mathrm{SOH};\; [0, 20, 40, 60, 80, 100],\; [20, 16, 13, 12, 10, 9]\big). \tag{38}$$

The reading: a fresh battery uses about 9% SOC per circuit; a heavily degraded battery uses about 20%.

The POH curve is generic; each aircraft uses slightly more or less SOC per circuit than the table predicts. We calibrate a per-plane scalar multiplier $k^\star$ by exploiting a structural fact: real flights fly an integer number of circuits. For any candidate k, the inferred number of circuits during event k is

$$\widehat{n}_k(k) \;=\; \frac{\Delta\mathrm{SOC}_k}{k\, s_{\mathrm{POH}}(\mathrm{SOH}_k)}, \tag{39}$$

and we pick the k that makes $\widehat{n}_k$ cluster most tightly around integers:

$$k^\star \;=\; \arg\min_{k}\; \frac{1}{n}\sum_{k=1}^{n}\Big|\widehat{n}_k(k) - \mathrm{round}\big(\widehat{n}_k(k)\big)\Big|. \tag{40}$$

The result is $k^\star_{166} = 0.80$ for Plane 166 (slightly more efficient than the POH baseline) and $k^\star_{192} = 0.74$ for Plane 192. The number of feasible circuits at start SOC S_0 with a 30% reserve is then

$$N_{\max} \;=\; \left\lfloor \frac{\max(S_0 - 30,\; 0)}{k^\star\, s_{\mathrm{POH}}(\mathrm{SOH})} \right\rfloor. \tag{41}$$

8.2 SOC Burn Rate

Circuits give a coarse view of capacity. For finer planning, we predict the SOC burn rate during flights:

$$r_k \;=\; \frac{\mathrm{SOC\ drop}_k}{\mathrm{duration}_k\ (\mathrm{minutes})}. \tag{42}$$

We fit a linear regression

$$\widehat{r}_k \;=\; \beta_0 + \sum_{j} \beta_j\, x_{k,j} \tag{43}$$

on a small set of physically meaningful features: latent SOH, mean and peak current, mean cell temperature, mean voltage, SOC range, current rate-of-change, and the BMS coulomb-gap signal. The linear form is chosen deliberately: SOC burn rate is much more directly observable than SOH, the relationship is approximately linear in the physically meaningful features, and we want the model to be interpretable.

The fit is strong. On the Plane 166 test split, the model achieves MAE = 0.19 SOC%/min and $R^2 = 0.56$; on Plane 192 (out of distribution) it achieves $R^2 = 0.54$. This is the most directly useful piece of the operational layer, because mission energy planning depends more on SOC burn than on SOH per se.

9 Putting It All Together

End to end, the system performs the following:

1. Parse raw telemetry into events. Reclassify event type from SOC direction so charge and flight events are correctly identified. Summarize each event into a single feature row.
2. Detect how trustworthy each event's SOH measurement is via bounded condition scores. Combine them into an event-specific measurement variance R_k.
3. Run a Kalman filter that treats latent SOH as a slowly drifting hidden state and observed BMS SOH as a noisy measurement of it. The filter automatically down-weights extreme events because their R_k is larger.
4. Use the causal filtered latent SOH as the supervised target for forecasting models. Train LSTM and GRU sequence models in delta mode with Huber loss and walk-forward validation. Use physics-informed variants (hybrid, PINN) where physical structure helps.
5. For long-horizon outlook, calibrate an isotonic backbone learned from an external eVTOL dataset that does see end of life, and place each of our batteries on that backbone via a two-parameter grid search.
6. Translate SOH into operational quantities: feasible circuits per flight (from a POH-derived map with per-plane calibration) and SOC burn rate (from a small linear regression on physically meaningful features).

10 Limitations

- Latent SOH is modeled as a scalar random walk. A two-state model with explicit degradation rate would be more expressive but requires more data per battery than we have.
- The measurement variance R_k is built from interpretable but heuristic stress scores. The weights were validated by checking that the filter residuals were well-behaved, not derived from a first-principles noise model.
- Short-horizon forecasts (one to five flights) are reliable. Beyond about ten flights, forecast error grows substantially, which is why the long-horizon outlook uses backbone calibration rather than the recurrent forecaster.
- Plane 192 has only 98 events; it is a useful sanity check but is too small to be a strong claim of cross-aircraft generalization.
- The PINN enforces an ODE, not a partial differential equation. A full electrochemical PINN (single-particle or Doyle–Fuller–Newman) would require dense voltage–current trajectories that we do not have.

11 Why Each Choice Was Made

Distilled to a single paragraph: raw BMS SOH was rejected as a label because empirically it jumps by tens of points in a single event. A Kalman filter was chosen because it is the simplest principled way to combine many noisy measurements of a slowly-drifting hidden quantity. The

filter is condition-aware because we know which events are unreliable, and ignoring that knowledge would waste it. Sequence models were chosen because aging depends on history, not just on present state. Physics-informed variants were added to constrain learning when data are limited. An external backbone was added because our aircraft data does not span a full battery lifetime. The operational layer was kept simple — linear models and lookup tables — because the quantities it predicts are directly observable and a complex model would not gain accuracy. Throughout, the goal was not to maximize model complexity but to make each step as transparent as the data would support.

References

[1] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 1. Background," *Journal of Power Sources*, vol. 134, no. 2, pp. 252–261, 2004. doi: 10.1016/j.jpowsour.2004.02.031. *(Foundational treatment of EKF for battery management. Establishes the conceptual frame we follow: battery state quantities are hidden, estimated from noisy terminal measurements.)*

[2] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 2. Modeling and identification," *Journal of Power Sources*, vol. 134, no. 2, pp. 262–276, 2004. doi: 10.1016/j.jpowsour.2004.02.032. *(State-space modeling and parameter identification for lithium battery packs in the Kalman framework.)*

[3] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 3. State and parameter estimation," *Journal of Power Sources*, vol. 134, no. 2, pp. 277–292, 2004. doi: 10.1016/j.jpowsour.2004.02.033. *(Treats SOH as a slowly varying parameter recovered through recursive Kalman estimation. Direct precedent for our random-walk latent-SOH formulation.)*

[4] H. E. Rauch, F. Tung, and C. T. Striebel, "Maximum likelihood estimates of linear dynamic systems," *AIAA Journal*, vol. 3, no. 8, pp. 1445–1450, 1965. doi: 10.2514/3.3166. *(Original paper deriving the backward-pass smoother we use for retrospective latent-SOH estimation.)*

[5] R. K. Mehra, "Approaches to adaptive filtering," *IEEE Transactions on Automatic Control*, vol. AC-17, no. 5, pp. 693–698, 1972. doi: 10.1109/TAC.1972.1100100. *(Classical reference for Kalman filtering with time-varying noise covariance. Establishes the legitimacy of adapting R_k, which our condition-aware noise model does in an a-priori, physics-driven way rather than from innovation statistics.)*

[6] S. Särkkä and L. Svensson, *Bayesian Filtering and Smoothing*, 2nd ed. Cambridge University Press, 2023. *(Standard modern textbook for Kalman filtering and RTS smoothing. Used as the methodological reference for the linear-Gaussian formulation and notation in this report.)*

[7] S. Yang, C. Zhang, J. Jiang, W. Zhang, L. Zhang, and Y. Wang, "Review on state-of-health of lithium-ion batteries: Characterizations, estimations and applications," *Journal of Cleaner Production*, vol. 314, p. 128015, 2021. doi: 10.1016/j.jclepro.2021.128015. *(Recent review surveying SOH estimation techniques. Confirms that adaptive Kalman variants with condition-dependent noise are an established branch of the SOH-estimation literature, consistent with our approach.)*

[8] Z. Yu, Y. Zhang, L. Qi, and R. Li, "SOH Estimation Method for Lithium-ion Battery Based on Discharge Characteristics," *International Journal of Electrochemical Science*, vol. 17, no. 7,

Article 220725, 2022. doi: 10.20964/2022.07.38. *(Uses an LSTM-based degradation model for battery SOH estimation under partial charging/discharging; supports our choice of recurrent sequence models for short-horizon forecasting.)*

[9] S. Jenu, A. Hentunen, J. Haavisto, and M. Pihlatie, "State of health estimation of cycle aged large format lithium-ion cells based on partial charging," *Journal of Energy Storage*, vol. 46, p. 103855, 2022. doi: 10.1016/j.est.2021.103855. *(Develops incremental-capacity and integrated-voltage SOH estimation from partial charging segments. Background for our charge-event coulomb-counting cross-check estimator.)*

[10] U. Saini and S. Renganathan, "Incremental capacity analysis of battery under dynamic load conditions," *MethodsX*, vol. 14, p. 103331, 2025. *(Extends ICA-style SOH estimation to electric-vehicle batteries under non-laboratory dynamic loads. Motivates our use of charge-event SOH only on clean, monotonic charge segments and our decision to treat raw BMS SOH as a noisy observation rather than ground truth.)*

[11] X. Gu, X. Huan, Y. Ren, W. Zhou, W. Jiang, and Z. Song, "Real-Time Physics-Aware Battery Health Monitoring from Partial Charging Profiles via Physics-Informed Neural Networks," arXiv preprint arXiv:2511.12053, 2025. *(Parameterizes a single-particle electrochemical model with a PINN to identify internal aging parameters from partial charging. Methodological inspiration for our physics-hybrid and ODE-form PINN forecasters, which adapt the idea of physics-informed losses to event-level aircraft telemetry rather than dense charge curves.)*

[12] A. Bills, S. Sripad, L. Fredericks, M. Guttenberg, D. Charles, E. Frank, and V. Viswanathan, "eVTOL Battery Dataset," Carnegie Mellon University, Pittsburgh, PA, USA. *(Public lithium-ion cycle-life dataset under representative eVTOL mission profiles. Provides the external long-life degradation shape that our isotonic backbone is fit to, enabling long-horizon remaining-life calibration despite our aircraft batteries not yet having aged to end of life.)*